CS781 Capstone Project: In-House Legal Intelligence System

Rohit Batra
261110054
[userid]@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

A law firm that cannot send client documents to external services needs legal NLP it can host itself. We built a seven-phase pipeline under that constraint. Phase 1 pretrains four compact language models from scratch on the official corpus (103–139M parameters: decoder-only Transformer, encoder–decoder, LSTM and dilated TCN) with our own 50K BPE tokenizer and an equal source mix. Every later model starts from these weights and adapts them, with LORMA, LoRA, IA^3, prefix and prompt tuning, adapters, layer-wise fine-tuning and knowledge distillation, to entity recognition, rhetorical roles, judgment prediction, statute identification, summarisation and prior-case retrieval. The decoder transferred best to five of six tasks. Reading the right part of long documents, a probe-then-fine-tune schedule that ended repeated training collapses, and per-label calibration moved the scores most. Distilled students two-thirds the size kept their teachers' quality, twice exceeding it.

1 Competition Result

Codabench username: `r_261110054` **Roll number:** 261110054

Table 1: Official phase scores and ranks (best valid submission per phase). Ranks recorded on 4 November 2026.

Phase	Task	Primary metric (value)	Rank	Valid submissions
1	Legal LM Pretraining	BERTScore F1 (0.8058)	133	4
2	Legal NER	Strict Macro-F1 (0.4912)	83	4
3	Rhetorical role prediction	Macro-F1 (0.5184)	47	7
4	Court judgment prediction	Macro-F1 (0.6795)	91	7
5	Legal statute identification	Macro-F1 (0.3037)	107	4
6	Abstractive summarization	BERTScore F1 (0.8354)	77	8
7	Prior case retrieval	Micro-F1@7 (0.2534)	0*	4

*The Phase-7 leaderboard was still changing when ranks were recorded, so no rank is given.

Total valid Codabench submissions across all phases: 38 **Date ranks were recorded:** 4 November 2026

We met the minimum of four distinct submissions in every phase; each covers all four Phase-1 backbones (Phase 7: three backbones and a distilled student). Extra runs in Phases 3, 4 and 6 were improved or decoding-only re-runs, so four or five distinct experiments are counted per phase (full list: `results/submissions.csv` in the repository).

Table 2: Data per phase and the analysis result that shaped the design.

Ph.	Train / dev / test	Finding → decision
1	4.34B tokens; 2% held out	High courts and tribunals are 91% of tokens (acts 3.5%) → sample the four sources equally.
2	70 (2 folds) / – / 35 docs; 8,393 spans	Only 3.8% of gold starts lie on a BPE token start (the leading space is part of the token) → predict boundaries per character.
3	80 / 10 / 10 docs; 17k sentences	Median sentence 32 tokens, documents up to 70k; Issue is 0.2% of CL → sentence encoder + document BiLSTM, class weights.
4	31.6k unique / 3,505 / 1,517	**All test cases also occur in dev, with labels** → never train or tune on them ("dev-clean", 1,988 cases). Outcome words differ only near the end → read the last chunks.
5	42.8k / 10.2k / 13.0k; 100 IDs	3.7 statutes per case; 42% of dev label sets unseen in train → independent sigmoids, per-statute thresholds.
6	7,030 / – / 100	Summaries are 22% of document length, drawn from the whole case; 4% novel words → summarise every chunk and join.
7	827 / 118 / 237 queries	Each query is in its own pool; median 4–5 relevant; BM25 scores 0.13 on full text, 0.33 on the first 2,000 words → exclude the query; use first and last chunks.

2 Introduction and Problem Description

Client confidentiality rules out sending judgments to a hosted language model, so a firm needs models it trains and runs itself, built here from only the official `Exploration-Lab/CS-781-Capstone` data. Phase 1 pretrains a language model of 50M–300M parameters from scratch; its test is to continue 1,000 legal prompts of 512 words with 128 words, scored by BERTScore F1 [1], ROUGE-L [2] and BLEU [3]. Phases 2–7 adapt that model to six tasks: character-level NER over 12 entity types (strict Macro-F1, so a span one character off is wrong); one of 13 rhetorical roles per sentence in Corporate Law and IT judgments (Macro-F1); binary judgment prediction (Macro-F1); multi-label identification of 100 statutes (Macro-F1 over statutes); headnote-style summarisation (BERTScore F1); and prior-case retrieval returning exactly seven candidates per query (Micro-F1@7). Every experiment starts from our own Phase-1 weights (no other pretrained model, external corpus or LLM API); each phase needs four distinct submissions including LORMA [4] and knowledge distillation; compute was free Kaggle T4 GPUs.

We asked whether four very different architectures trained on the same budget transfer differently, which parameter-efficient technique suits which backbone, how to represent documents of tens of thousands of tokens, how much decision calibration matters relative to training, and whether a distilled student keeps its teacher's quality. Phases were deliberately not pipelined into one another, so each stays a clean comparison of transfer from Phase 1.

3 Data Analysis and Preprocessing

3.1 Sources and official splits

We used only the official phase folders, downloaded inside each notebook. Table 2 gives sizes and the analysis finding that changed each design. Phases 2 and 6 have no official dev set, so we held out 14 of 70 and 150 of 7,030 documents.

3.2 Corpus statistics that informed design

A uniform Phase-1 sample would be dominated by High Court style, so each training window draws a source with equal probability (acts upsampled about 7×). NER labels are skewed (DATE 1,575 spans, R.COUNSEL 184; median span 13 characters, maximum 233). Rhetorical roles follow a

document order (facts, arguments, precedents, ratio, ruling), and CL dev has far more argument sentences than IT (41% vs 12%). In Phase 4, train is 59/41 while dev and test are balanced; "set aside" appears in the last 1,000 characters of 25% of label-1 and 6.6% of label-0 cases, with equal rates in the first 1,000. Statute frequencies span 43× (272 to 11,730 cases) and pairs co-occur strongly ($P(28 \mid 20) = 0.93$). In Phase 7 the oracle ceiling with exactly seven answers is only 0.62, because 13% of dev queries have one relevant case and a third have more than seven.

3.3 Preprocessing decisions

Text was kept cased (38K of 50K tokens contain capitals), except in Phase 4, whose dev and test splits are lower-case. Masked placeholders (3–5 BPE tokens each) became one plain word (`<SECTION>`→"Section") or were removed where they also hide ordinary words (`<ENTITY>`).

Phase-2 whitespace was never normalised: each 512-token window keeps a character-to-token map from the tokenizer's offsets, and decoding the gold boundaries through this path reproduces all 1,861 validation spans exactly. Phase 3 used the provided sentence lists unchanged; Phases 4–7 split cases into 510-token chunks (sentence-aligned 1,200-token chunks in Phase 6) and select chunks per task (Section 4).

4 Approach and Model Evolution

4.1 Phase-1 foundation model

We trained our own byte-level BPE tokenizer (vocabulary 50,000; 25M characters per source; `[PAD]` `[UNK]` `[BOS]` `[EOS]`), so legal tokens such as "Section" and "appellant" get their own entries.

The four architectures are A, a pre-LayerNorm decoder-only Transformer [5] (12 layers, d=768, 12 heads, GELU FFN 3,072, learned positions, context 2,048, tied embeddings; 125.0M); B, a 6+6-layer encoder–decoder with shared embeddings that predicts the next 1,024 tokens from the previous 1,024 (139.3M); C, a three-layer LSTM [6] with 1,280 units (103.4M); and D, a TCN [7] of 10 residual blocks of weight-normalised causal convolutions with dilations 1–512 and a 4,093-token receptive field (114.2M). Each notebook asserts its parameter count lies within 50M–300M. All four were trained with next-token cross-entropy on 100M tokens, about 2.3% of the corpus and the most the Kaggle sessions allowed. We used AdamW ($\beta = (0.9, 0.95)$, weight decay 0.1), a learning rate of 3×10^{-4} with cosine decay to 3×10^{-5}, 16K tokens per step and bf16, and selected checkpoints on the held-out loss.

Decoding produced exactly 128 words, near-greedy for A and C (temperature 0.1, top-p 0.95), temperature 0.7 with repetition penalty 1.1 for B, and temperature 0.8 for D. The untuned near-greedy setting made A repetitive: A had the lowest perplexity (20.3, against 31.8, 48.0 and 48.2) but the lowest BERTScore (0.793 against 0.806, 0.802 and 0.805).

4.2 Transfer protocol (Phases 2–6, and 7)

Every downstream model initialises from a Phase-1 checkpoint and adds a task head (Figure 1); backbone weights stay frozen unless the technique tunes them.

We implemented full and layer-wise fine-tuning, BitFit [8], LoRA [9], IA^3 [10], prefix [11] and prompt tuning [12], Houlsby and bottleneck adapters [13], knowledge distillation (KD) [14], and LORMA [4], a multiplicative update $W_{\text{eff}} = (I + \frac{\alpha}{r} BA)\, W_0$ with r=8, α=16 and B=0 at the start, applied to q/k/v/out/fc1/fc2 of A and to the recurrent gate matrices of C.

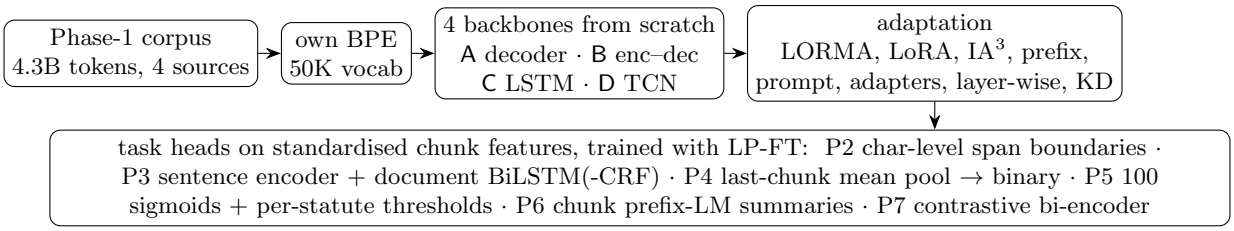


Figure 1: System overview: one set of Phase-1 backbones, separately adapted copies for each phase.

Two pieces of infrastructure carry across phases. First, A's residual stream has a few nearly constant dimensions ($|x| \approx 46$ against a mean of 2), so chunk features are *standardised per dimension*. Second, from Phase 4 on we used *LP-FT* [15]: a probe (or retrieval projection) fit on cached frozen features becomes "epoch 0", adapter and head are then fine-tuned at 10^{-4}, and the best epoch on validation is kept.

4.3 Phase 2 — Legal NER

Since almost no gold start lies on a token start, we built a character-level span head: each character receives its token's backbone state plus a 32-dimensional character embedding, a small convolution follows, and two heads give start and end probabilities for each label. Training uses binary cross-entropy with a positive weight capped at 30 (boundaries are 0.01–0.08% of characters) and output biases initialised from label prevalence; 512-token windows overlap with stride 384 and are averaged in character space. Decoding pairs each start above its label's threshold with the best end within a per-label maximum length, then resolves overlaps; the thresholds and caps were tuned on validation. The best system, C with LORMA on the recurrent weights, trains 0.79M parameters; an earlier additive adapter (not true LORMA) scored 0.372 against 0.392 before calibration.

4.4 Phase 3 — Rhetorical role prediction

Each sentence ($\leq$128 tokens) is encoded into [mean; final state], projected to 512 dimensions with a domain embedding and relative position; a two-layer document BiLSTM over 128-sentence windows adds context, and from S7 on a linear-chain CRF [16] decodes roles with Viterbi. CL and IT are trained jointly (40 documents each is too few) with class weights $(N/Kn_c)^{0.5}$ capped at 10.

The best submission (S6) is a KD student keeping layers 0, 2, ..., 10 of A (DistilBERT-style [17]) with LoRA, trained with CRF NLL, weighted cross-entropy and $T^2 \cdot \mathrm{KL}$ ($T = 2$) against the S7 teacher, then retrained on train+dev for the epoch count chosen on dev.

4.5 Phase 4 — Court judgment prediction

Each case is represented by its last two chunks of 510 tokens. A chunk vector [mean; final state] is standardised, the chunk vectors are averaged, and a linear layer gives the two-class output. Train is 59/41 while test is balanced, and a majority-class predictor scores ≈0.33 Macro-F1 (what our collapsed runs produced); class-weighted cross-entropy and a threshold tuned on dev-clean (0.41) handle the shift.

4.6 Phase 5 — Legal statute identification

Up to 8 chunks are selected: the first, the last, and those with the most "Section" or "Act" tokens. Each case is pooled as [mean; max] over chunks and passed to 100 independent sigmoids, trained with binary cross-entropy whose positive weight is $\sqrt{\mathrm{neg}/\mathrm{pos}}$, capped at 10. Every statute has at

least 68 dev positives, so each gets its own threshold; we report cross-fitted estimates (tuned on half of dev, scored on the other half) and give every case at least one statute. Rare statutes are handled by the capped positive weight and per-statute thresholds rather than resampling.

4.7 Phase 6 — Abstractive summarisation

Only 19% of documents fit in 2,048 tokens, so we use sentence-aligned 1,200-token chunks. Each summary sentence is assigned to the chunk that contains its best-matching source sentence (89% match with word recall ≥ 0.5). A with LORMA is trained as a prefix language model on `[BOS] chunk ‘‘Summary:’’ target [EOS]`. At inference every chunk is summarised with a KV cache and 4-gram blocking, and the chunk summaries are joined with duplicate sentences removed.

The original decoding was short and repetitive (20–39% repeated trigrams); a grid on 60 validation documents chose a minimum length of 22% of each chunk and a CTRL repetition penalty [18] of 1.2. The tuned runs replaced the originals.

4.8 Phase 7 — Prior case retrieval

We built a bi-encoder in the style of dense retrieval [19]. Each chunk's [mean; final state] features are standardised and projected to 256 dimensions; the case vector is the normalised mean over the first 8 and last 8 chunks, and candidates are ranked by cosine similarity with the query removed from its own ranking. Training uses InfoNCE [20] at temperature 0.05 over one relevant case, two hard negatives from the TF-IDF top 50 and all in-batch documents. BM25 [21] and TF-IDF only served as references and negative miners. The KD student (S4) keeps 6 of A's 12 layers and matches the teacher's in-batch similarity distribution ($T^2 \cdot \mathrm{KL}$, $T = 2$), plus one teacher-mined hard negative per pair.

4.9 Final submitted system

The submitted systems are B in Phase 1, C+LORMA S3 with the calibrated decoder in Phase 2, the S6 student (stage 2) in Phase 3, and A+LORMA S1 in Phases 4–7 (Phase 6 with tuned decoding); Table 3 gives their settings, and each checkpoint is a Kaggle dataset linked from the phase README.

5 Experiments and Implementation Details

5.1 Compute and training protocol

All runs used Kaggle: a single T4 in Phase 1 and two T4s with DataParallel later, 30–160 minutes per run, and 46 training notebooks in total. Notebooks save `latest.pt` (with optimiser, scheduler and sampler state) every few hundred steps, so a run interrupted by a session limit resumes exactly.

Downstream runs used AdamW (weight decay 0.01), linear warm-up over 3–6% of steps then linear decay, clipping at 1.0 and fp16; checkpoints were selected on the validation task metric (validation loss in Phase 6).

5.2 Local evaluation

Every phase has a labelled held-out set that never touches test labels: 2% of documents held out by hash (P1), 14 of 70 labelled documents (P2), the official dev sets (P3, P5, P7), dev minus the 1,517 test cases (P4), and 150 training documents (P6). Each notebook implements the official

Table 3: Hyperparameters of the final model in each phase (total parameters; trainable in brackets).

	P1 B	P2 C+LORMA	P3 KD student	P4 A+LORMA	P5 A+LORMA	P6 A+LORMA	P7 A+LORMA
Params (M)	139.3	104.2 (0.79)	85.5 (3.04)	126.4 (1.4)	126.7 (1.63)	126.4 (1.33)	126.7 (1.72)
Context	1k→1k	512/384	128/sent.	2×510	≤8×510	1,200+640	16×510
Peak LR	3e-4	2e-4	3e-4/1e-3	1e-4	1e-4	2e-4	1e-4
Batch×acc	1×8	2×1	128 sent.	8 cases	16 ch.×2	4×4	4 pairs
Epochs	6,104 steps	3 (best 2)	20 (best 17)	4 (best 3)	3	2	3 (best 2)
Dropout	0.1	0 (frozen)	0.2 head	0	0.1 head	0	0
Decode/thr.	T0.7, rep 1.1	thr.+caps	Viterbi	thr. 0.41	per-statute	22%, rep 1.2	top-7 cosine

Table 4: Controlled ablations on validation (Phase 6 in ROUGE-1).

Ph.	Change (all else fixed)	Δ	Ph.	Change (all else fixed)	Δ
2	per-label span caps + thresholds	+0.102	5	first chunk only vs evidence chunks	−0.035
3	document BiLSTM vs sentence-only	+0.158	5	thresholds scored on tuning data	+0.015 bias
3	Viterbi CRF vs argmax	+0.042	6	all chunks joined vs first chunk	+0.158
3	prefix tuning vs LoRA (test)	−0.049	6	tuned decoding, same weights	+0.060
4	last 2 vs first 2 chunks	+0.102	7	contrastive proj. on frozen C	+0.074
4	LORMA fine-tuning over its probe	+0.088	7	mean vs max pooling over chunks	+0.089
4	4 epochs vs 2 (same recipe)	+0.024	7	first 8+last 8 vs first chunk	+0.157

metric and validates `prediction.json` (all and only test IDs, exact types, per-phase constraints such as seven unique pool IDs excluding the query).

Matching the official metric exactly mattered: in Phase 3, the first S1 score used scikit-learn's Macro-F1, which averages only over labels present, and reported 0.476 against Codabench's 0.402, the 13-label average we used from then on. Local and test scores agree closely (plot in `results/plots`): the cross-fitted Phase-5 estimates were within ±0.01 of test for all four runs. The largest gaps came from the smallest sets (10 documents in Phase 3; 118 queries in Phase 7, where every run lost 0.02–0.04).

5.3 Ablations and controlled comparisons

Table 4 changes one factor at a time. Document structure and decision calibration produced the largest effects, larger than switching adapters on a fixed backbone; prefix tuning lost to LoRA under an identical pipeline, and max pooling lost badly to mean pooling for retrieval.

6 Results

Table 5 gives every counted submission. In Phase 1, B had ROUGE-L 0.162 and BLEU-4 0.015 (best: D, 0.185 and 0.021). Official secondary metrics of the best runs were precision 0.662 / recall 0.744 (P4, label 1), Micro-F1 0.399 (P5), ROUGE-1 0.527 / ROUGE-L 0.240 (P6) and precision@7 0.259 / recall@7 0.248 (P7).

Which system won, and why. A adapted with LORMA (or distilled from it) won five of six downstream phases (Figure 2), as frozen-feature probes predicted before any fine-tuning (Phase-3 sentence probes: A 0.283, C 0.235, D 0.226, B encoder 0.117; frozen Phase-7 retrieval: A 0.238, D 0.122, C 0.112).

A had seen the longest contexts and had the lowest perplexity, so its states encode more of the document, and LORMA reshapes every projection slightly while touching about 1% of the parameters. NER is the exception: a boundary decision is local and character-precise, and the LSTM state, adjusted multiplicatively on the recurrent gates, gave the sharpest boundaries.

Table 5: Counted submissions: local validation and official test scores, in the order listed (best per phase in bold; trainable parameters in `results/submissions.csv`).

Ph.	Submissions: backbone + technique	Local	**Test**
2	S1 A full FT / S2 B enc.+LoRA / **S3 C+LORMA** / S4 KD D→6-block D	.225 / .198 / .494 / .220	.192 / .177 / **.491** / .253
3	S1 C+LORMA / S3 B+IA^3 / S4 D+prompt / S7 A+LoRA+CRF / **S6 KD S7→6-layer A**	.40 / .219 / .329 / .450 / .458	.402 / .236 / .339 / .502 / **.518**
4	**S1 A+LORMA LP-FT** / S2 C layer-wise / S3 B BitFit / S4 D adapters / S5 KD S2→2-layer C	.695 / .562 / .531 / .578 / .613	**.680** / .562 / .491 / .527 / .592
5	**S1 A+LORMA** / S2 C layer-wise / S3 D adapters / S4 KD ensemble→B+LoRA	.298 / .273 / .271 / .215	**.304** / .263 / .273 / .220
6	**S1 A+LORMA** / S2 B+LoRA / S3 D Houlsby / S4 KD A→C (tuned decoding)	R1 .521 (S1)	**.835** / .809 / .814 / .816
7	**S1 A+LORMA** / S2 C layer-wise / S3 D prompt / S4 KD S1→6-layer A	.269 / .217 / .198 / .280	**.253** / .180 / .181 / .237

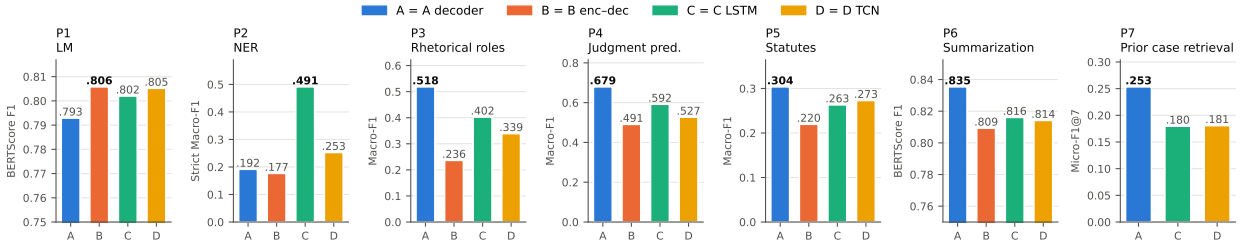


Figure 2: Best official score per backbone and phase (KD students under their own backbone; Phase 6 with tuned decoding; no B run in Phase 7; BERTScore axes start at 0.75). A wins Phases 3–7; C wins NER.

Learning curves. In the LP-FT runs (Figure 3) the probe (epoch 0) recovers most of the score and fine-tuning adds 0.02–0.09 (Phase-1 per-step curves were not exported).

Per-class results. In NER (S3, validation) DATE (0.91), COURT (0.77) and STAT (0.73) are reliable; APP, AUTH, RESP, WIT and PREC (0.25–0.30) are weak. In rhetorical roles, Statute (0.80) and RulingByPresentCourt are easy, Issue and None stay weak, and ArgumentRespondent recall rose from 0.26 to 0.54 only with the CRF and KD. In statutes, per-statute F1 rises with training frequency, from 0.20 in the rarest quartile to 0.45 in the most frequent (rank correlation ≈0.6).

7 Error Analysis and Discussion

7.1 Phase-wise failure modes

Phases 1–3. Near-greedy A fell into repetitive loops; all four models' continuations were legal in style but drifted from the prompt's topic, compressing BERTScore to a 0.013 range. In NER, the uncalibrated S3 model predicted spans up to 512 characters (gold spans are under 150) until per-label caps removed this; AUTH is over-predicted (436 false vs 105 true positives), and APP vs RESP hinges on a position relative to "versus" often far from the name. Rhetorical-role errors concentrate where the role depends on what the court later does with a sentence (is a precedent relied upon?) or on who is speaking, and on rare roles (Issue has 15 dev sentences).

Phase 4. A with LORMA collapsed to a constant prediction in four runs. Removing one suspected cause per run (attention pooling, outlier dimensions via standardisation, dropout in the frozen backbone) did not help: the cause was a randomly initialised head combined with multiplicative updates, which destroy the features before the head finds the weak signal. LP-FT fixed it (probe 0.606, fine-tuned 0.695). Remaining errors favour acceptance (recall 0.744, precision

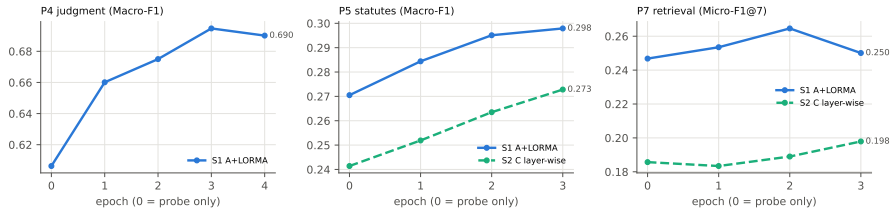


Figure 3: Validation metric vs epoch for LP-FT runs; epoch 0 is the probe or projection alone.

0.662): with the ruling removed, partly allowed or remanded cases read like acceptances.

Phase 5. The dominant error replaces a specific statute with a frequent generic one: statutes 12, 13 or 14 missed while 5 is predicted, and statutes 34, 77 or 61 missed while 97 is predicted (the first and third most frequent): with references masked, the model falls back on what is typical.

Phases 6–7. Chunk summaries are written independently, so facts are restated and the holding is sometimes split; longer tuned outputs raised ROUGE-1 and BERTScore but lowered ROUGE-L for three of four models. In retrieval, precision@7 is 0.05–0.08 for queries with one relevant case, and recall is only 0.09–0.13 for queries with 16 or more. S1 finds nothing for 18 of 118 dev queries. All four rankers stay below TF-IDF (0.368 on dev), but their hits differ: S1 and TF-IDF together find 393 relevant cases, against 331 for TF-IDF alone.

7.2 What the metrics miss, limitations and next steps

BERTScore and ROUGE reward similarity to one reference, not whether the holding is stated correctly, so a fluent summary that misstates the outcome can still score well. Strict span-F1 gives no credit for a correctly typed entity one character off; Macro-F1 weights a statute seen 272 times as much as one seen 11,730 times; Micro-F1@7 caps any system at 0.62 on dev. The scores rank systems; they do not certify them. Each Phase-1 model saw 2.3% of the corpus and three were still improving; fine-tuning was held to 2–4 epochs; there was no human legal evaluation.

With more compute we would pretrain A on 1B tokens with a 4,096 context; add contrastive pretraining on citing–cited pairs built from Phase-7 training labels only; use gradient caching to raise in-batch negatives from about 15 to several hundred; distil one A-family student for all tasks; and mix neural and lexical scores, which the complementary hits support.

7.3 Surprising findings

The best-perplexity model had the worst Phase-1 BERTScore yet became the best downstream backbone. KD students beat their teachers in Phase 3 (0.518 vs 0.502 at 66% of the parameters) and Phase 4 (0.592 vs 0.562), and in Phase 7 on dev (0.280 vs 0.269, 1.99× faster): with little labelled data, soft targets regularise more than they compress. In Phase 6, two decoding numbers helped every model more than any architectural change.

8 Conclusion

We delivered a self-hosted legal NLP stack (own tokenizer, four foundation models, six adapted task models on one 16 GB GPU). Reading the right part of each document, stable optimisation and decision calibration mattered more than the adapter choice; backbone quality, visible in frozen probes, decided most outcomes. A skeptical partner should not yet trust unreviewed summaries, cue-based judgment predictions, or a retriever below word matching.

LLM Usage Declaration

Q1. Which Large Language Model(s), if any, did you use while completing the competition or writing this report?

Answer: Claude (Anthropic), used through the Claude app as an AI assistant from Phase 3 onwards, and for the documentation and this report. [Add any other LLM you used, including for the Phase 1–2 notebooks.]

Q2. For each row, mark Yes or No and state the purpose in one short phrase.

Part	Used LLM?	Purpose of using the LLM
Understanding concepts / papers	Yes	Explanations of LoRA, LORMA, LP-FT, KD and retrieval losses
Ideation / brainstorming at a high level	Yes	Proposing per-phase plans and techniques for each backbone
Debugging small ancillary snippets	Yes	Diagnosing errors from notebook outputs (e.g. the Phase-4 collapse)
Plotting / visualization code	Yes	Wrote `report/make_figures.py`
Grammar / writing style of the report	Yes	Drafted and edited the report text
Report structure / section wording	Yes	Structured the report and repository READMEs
Designing the model architecture	Yes	Designed the Phase 3–7 task heads and adaptation setups [state Phase 1–2]
Hyperparameter search / tuning	Yes	Chose settings and the decoding-tuning grid (Phase 6)
Analysis of results / conclusions	Yes	Drafted the README analyses and this report's analysis
Writing the core training / inference code	Yes	Wrote the Phase 3–7 notebooks [state Phase 1–2]
Other (specify)	Yes	Data-analysis notebooks, repository documentation

Q3. Did you refer to sources other than lecture slides and an LLM?

Answer: Yes. The papers in the bibliography (LoRA, LORMA, LP-FT, IA^3, prefix and prompt tuning, adapters, BitFit, KD, DistilBERT, CTRL, InfoNCE, DPR, TCN, CRF, BM25), the capstone document, the official dataset and its starter notebook, and the Codabench evaluation output. [Add any other source you used.]

Q4. Apart from any permitted LLM use, what was *your* contribution?

[To be written by the student in their own words (8–12 sentences), including at least two things tried that failed.]

References

[1] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating text generation with BERT," *International Conference on Learning Representations*, 2020.

[2] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*, pp. 74–81, 2004.

[3] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: a method for automatic evaluation of machine translation," in *Proceedings of ACL*, pp. 311–318, 2002.

[4] H. Bihany, S. Patel, and A. Modi, "LoRMA: Low-rank multiplicative adaptation for LLMs," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025.

[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[6] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[7] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," *arXiv preprint arXiv:1803.01271*, 2018.

[8] E. Ben Zaken, S. Ravfogel, and Y. Goldberg, "BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models," in *Proceedings of ACL*, 2022.

[9] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.

[10] H. Liu, D. Tam, M. Muqeeth, J. Mohta, T. Huang, M. Bansal, and C. Raffel, "Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning," in *Advances in Neural Information Processing Systems*, 2022.

[11] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," in *Proceedings of ACL-IJCNLP*, 2021.

[12] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," in *Proceedings of EMNLP*, 2021.

[13] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for NLP," in *International Conference on Machine Learning*, 2019.

[14] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[15] A. Kumar, A. Raghunathan, R. Jones, T. Ma, and P. Liang, "Fine-tuning can distort pretrained features and underperform out-of-distribution," in *International Conference on Learning Representations*, 2022.

[16] J. Lafferty, A. McCallum, and F. Pereira, "Conditional random fields: Probabilistic models for segmenting and labeling sequence data," in *International Conference on Machine Learning*, 2001.

[17] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.

[18] N. S. Keskar, B. McCann, L. R. Varshney, C. Xiong, and R. Socher, "CTRL: A conditional transformer language model for controllable generation," *arXiv preprint arXiv:1909.05858*, 2019.

[19] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of EMNLP*, 2020.

[20] A. van den Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," *arXiv preprint arXiv:1807.03748*, 2018.

[21] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.